

3

Artificial Neurons, Neural Networks and Architectures

Floating on neural activity, the symbol level of the brain mirrors the world. But neural activity, which can be simulated on a computer does not create thought; that calls for higher levels of organization.

Douglas Hofstadter

3.1 Neuron Abstraction

Our discussion in Chapter 2 focussed on the microscopic behaviour of real neurons as we now understand them. Real neurons integrate hundreds or thousands of temporal signals through their dendrites. These signals modify their internal potential in a complex way that depends on the inhibitory or excitatory nature of the synapses at which the signals impinge on the cell. They transmit signals in the form of an action potential when their internal cell potential at the axon hillock exceeds a threshold of about -40 mV with respect to the external medium.

At synapses, neurons transduce signals—electrical to chemical, and then from chemical back again to electrical. This change in the form of signals takes place with the help of complex neurotransmitters and protein based ion-specific channels. Neurotransmitters released into the extra-cellular medium selectively open or close channels in the postsynaptic membrane, thus modulating the internal cell potential of the postsynaptic neuron. Since the entire neurotransmitter released is not utilized, each synapse is associated with what we call the *synaptic efficacy*—the efficiency with which a signal is transmitted from the presynaptic to postsynaptic neuron.

Action potentials are signals in the space-time continuum of axons.

3.1.1 Neuron Activations

We draw upon the neuron metaphor in order to integrate important principles of the working of real neurons into a simple mathematical neuron model. Figure 3.1 shows the j th artificial neuron that receives input signals s_i from possibly n different sources. These signals traverse weighted pathways w_{ij} .

The neuron abstraction considered by artificial neural networks represents a drastic simplification of real neurons. More recently, however, neural networks have begun to move closer to neurobiology with the introduction of pulsed neural networks which we will study in Chapter 13.

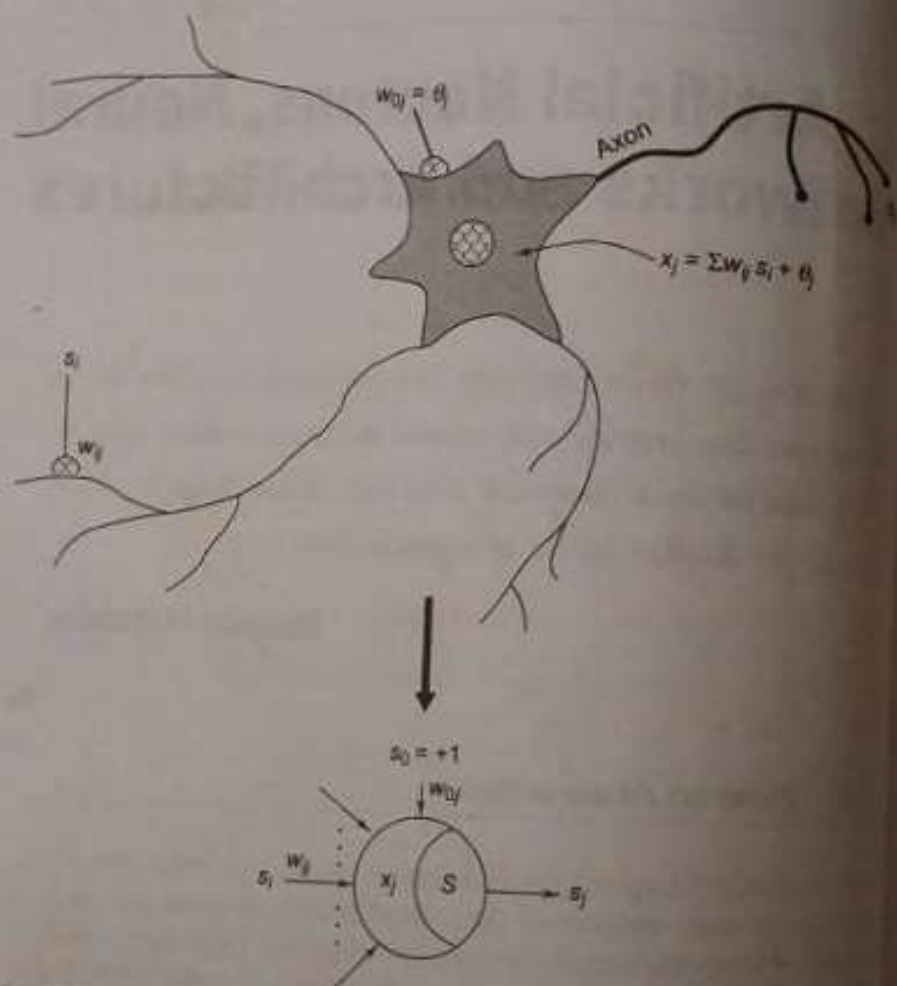


Fig. 3.1 Artificial neuron model—drawn to remind us of its biological origin

in order to generate an internal activation x_j , which is a linear weighted aggregation of the impinging signals, modified by an internal threshold:

$$x_j = \sum_{i=1}^n w_{ij} s_i + \theta_j \quad (3.1)$$

Here, connection weights w_{ij} model the synaptic efficacies of various inter-neuron synapses. Positive weights correspond to excitatory synapses, while negative weights model inhibitory synapses. Impinging signals s_i represent

Notation: w_{ij} denotes the weight from neuron i to neuron j . A number of texts use the reverse convention.

mathematical abstractions of action potentials. The threshold θ_j represents the internal firing threshold of a real neuron, and the activation x_j then models the internal aggregated cell potential.

The activation of the neuron is subsequently transformed through a *signal function* $S(\cdot)$, to generate the output signal $s_j = S(x_j)$ of the neuron. As we discuss in detail in Section 3.2, a signal function may typically be *binary threshold*, *linear threshold*, *sigmoidal*, *Gaussian*, or *probabilistic*.

For the sake of notational convenience we often consider the threshold θ_j as an additional weight $w_{0j} = \theta_j$ fanning into the neuron with a constant input $x_0 = +1$ (see Fig. 3.1).

In biological systems, cell potentials change faster than changes in synaptic efficacies. In the brain, a thought or a reasoning process flashes across neuron ensembles in the form of fast changing patterns of internal cell potentials and action potentials. In other words, neuron activations x_j , change quickly as neurons sample and integrate impinging signals. However, the learning process—where impinging information is downloaded into memory by changing synaptic efficacies—takes place on a much longer time scale. Learning concerns itself with changes in weights w_{ij} in response to stable patterns of activity, and is a slow process. In neural networks changes in synaptic weights are implemented using a *learning algorithm*. We often refer to the rapidly changing x_j as a *short term memory* (STM) and to the slowly changing w_{ij} as *long term memory* (LTM).

This representation lends convenience in the formulation of learning algorithms which we will study in forthcoming chapters.

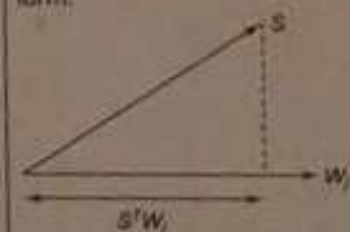
3.1.2 Activations Measure Similarities

Equation (3.1) admits a linear algebraic interpretation of the process of activity aggregation at an artificial neuron. The activation x_j is simply the inner product of the impinging signal vector $S = (s_0, \dots, s_n)^T$, with the neuronal weight vector $W_j = (w_{0j}, \dots, w_{nj})^T$:

$$x_j = S^T W_j = \sum_{i=0}^n w_{ij} s_i \quad (3.2)$$

where $s_0 = +1$ and $w_{0j} = \theta_j$. This inner product represents a similarity measure, or the degree of match, between the input signal vector and the weight vector. In other words, the activation of a neuron measures the "similarity" between the input signal vector and the neuronal weight vector of the neuron in question. This formalism allows us to view the neuron as a *linear filter* in the sense that the neuron is able to discriminate between inputs that are similar to its weight vector (which tend to generate large activations), and inputs that are dissimilar to the neuron weight vector (which tend to generate small activations). Notice that if S is at right angles with W_j , the projection of S on W_j and therefore x_j is zero. Such *orthogonal* vectors will generate zero inner-products and thus zero activations. On the other hand, an input vector completely aligned with the weight vector will generate a maximal activation, and one that is aligned but in the opposite direction will

Note: Throughout we assume that vectors are in columnar form.



The closer S is to W_j , the larger the projection or inner product or activation.

generate a maximally negative activation. Always keep these points in your mind.

We have seen that a neuron filters its inputs with the help of the inner product, and measures the input-to-weight vector similarity or dissimilarity with the value of its activation. As Grossberg points out, when endowed with learning capabilities the neuron thus acquires the status of being an *adaptive filter* [194].

3.2 Neuron Signal Functions

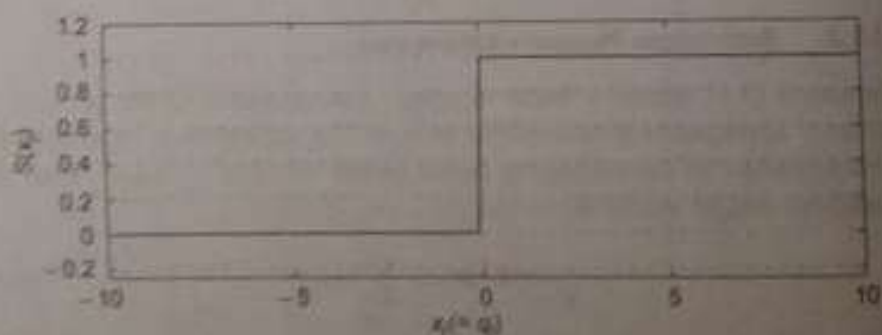
3.2.1 Binary Threshold Signal Function

Neurons with binary signal functions are loosely referred to as binary neurons or threshold logic neurons (TLNs) and they find their origin in the 1943 publication of McCulloch and Pitts [379]. As before, a binary neuron computes its activation, $x_j = S^T W_j$, and generates a +1 or a 0 signal depending upon whether this activity is positive or negative:

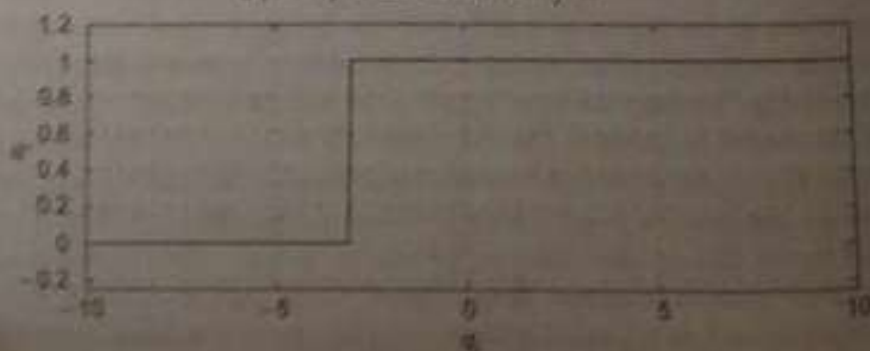
$$\delta(x_j) = \begin{cases} 1 & x_j \geq 0 \\ 0 & x_j < 0 \end{cases} \quad (3.3)$$

As Fig. 3.2(a) shows, this translates to saying that net positive activations

The signal function (Eq. 3.3) is none other than the all familiar unit step function $u(\cdot)$ encountered in electrical engineering. We will encounter such signal functions in Chapters 4, 10, and 11.



(a) Binary threshold function: $\theta = 0$



(b) Binary threshold function: $\theta = +3$

Fig. 3.2 Binary threshold neuron signal function with different neuronal thresholds as indicated

translate to a +1 signal value, and net negative activations translate to a 0 signal value. The threshold logic function is a two-state machine, i.e. $s_j = S(x_j) \in \{0, 1\}$.

Notice that to get around the discontinuity at 0, Eq. 3.3 employs the "greater than or equal to" condition. An alternative solution to handling the discontinuity is to assume that the activation $x_j = 0$ is ambiguous and cannot be suitably translated to any value. In such a situation we might be justified in leaving the neuron signal unchanged. To model such a situation we assume that a neuron updates its signal at discrete instants of time, k , by sampling its instantaneous activation x_j^k . In *discrete time*, we may therefore write this modified binary threshold signal function as:

$$S(x_j^{k+1}) = \begin{cases} 1 & x_j^{k+1} > 0 \\ S(x_j^k) & x_j^{k+1} = 0 \\ 0 & x_j^{k+1} < 0 \end{cases} \quad (3.4)$$

Notation reminder: subscript j denotes the neuron index; the superscript k denotes a discrete instant of time.

where the updated signal value $S(x_j^{k+1})$ at time instant $k+1$ is generated from the neuron activation x_j^{k+1} , sampled at time instant $k+1$.

Finally, we make the point that the behavioural response of the threshold logic neuron as a two-state machine can be extended to the *bipolar* case where the signals are $s_j \in \{-1, 1\}$ instead of $\{0, 1\}$. The resulting signal function is then none other than the *signum function*, $\text{sign}(x)$ commonly encountered in communication theory.

$$S(x_j) = \begin{cases} +1 & x_j > 0 \\ -1 & x_j < 0 \end{cases} \quad (3.5)$$

We encounter an example of such a signal function when we study attractor neural networks in Chapter 10.

3.2.2 Interpretation of Threshold

It is important to understand the physical meaning of the addition of the threshold term θ_j into the activation equation (Eq. 3.1). We discuss this in the context of the binary threshold signal function, although the general idea carries over to other signal functions as well. We rewrite Eq. 3.1 as:

$$x_j = \sum_{i=1}^n w_{ij} s_i \quad (3.6)$$

$$= \sum_{i=1}^n w_{ij} s_i + w_{0j} \quad (3.7)$$

$$= q_j + \theta_j \quad (3.8)$$

where $q_j = \sum_{i=1}^n w_{ij} s_i$ is the activation that aggregates from *external* sources and θ_j is the threshold *internal* to the neuron. Think of q_j as the net external input to the neuron, or simply the activation of the neuron when the threshold is set to zero. From the point of view of the activation x_j , the

When we write $[a, b]$ it means that the range includes both a and b ; (a, b) is an example of a range that excludes a but includes b . Stand by for a detailed review of math terms used in the text in Section 3.3.

signal function is always centred at 0 as Fig. 3.2(a) shows, and $x_j = 1$ if $x_j \geq 0$, and 0 otherwise. The range of activation values that yields a 0 signal is $(-\infty, 0)$ and the range that yields a signal value of 1 is $(0, \infty)$.

However, if we look at the signal function from the point of view of the net external input q_j , we see that it shifts either right or left depending on the value of the internal threshold. For example, the signal function is shifted left by 3 in Fig. 3.2(b) where $\theta_j = 3$. In other words, the threshold biases the neuron to now fire a signal 0 for net external inputs in the range $(-\infty, -3)$, and a signal 1 for activations in the range $[-3, \infty)$. Often, θ_j is referred to as the bias—it biases the 0/1 firing ranges of the neuron for a given value of q_j . By allowing the signal function to shift on the q_j axis, the threshold gives the neuron an extra degree of freedom which (as we study ahead) essentially extends the learning capabilities of the neuron.

Another way to look at it is that from the point of view of the net activation x_j , the signal is +1 if $x_j = q_j + \theta_j \geq 0$, or $q_j \geq -\theta_j$, and is 0 if $q_j < -\theta_j$. The neuron thus "compares" the net external input $q_j = \sum_{i=1}^n w_{ij}x_i$ with $-\theta_j$; if q_j is greater than the negative threshold, it fires +1, otherwise it fires 0. These ideas are summarized in Fig. 3.3.

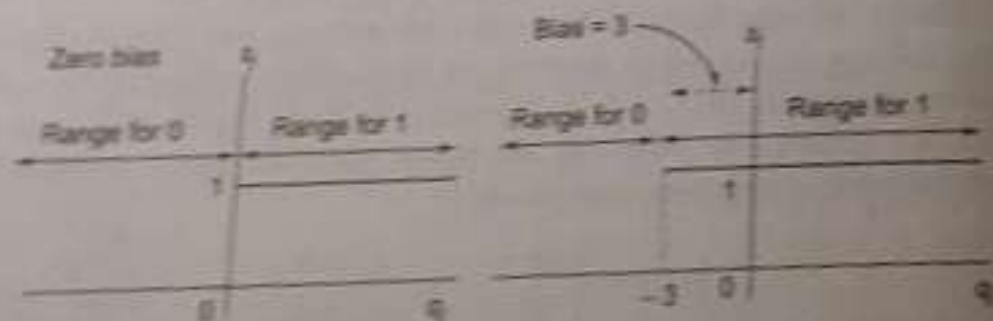


Fig. 3.3 The neuron threshold biases the range of values of the net input for which it fires a 0 or a +1 signal

Note that the addition of a positive threshold to the net input of the neuron is equivalent to shifting the threshold logic signal function to the left along the q_j axis by the threshold value. Similarly addition of a negative threshold results in a right-shift of the signal function on the q_j axis. These ideas will be reinforced in our subsequent discussion.

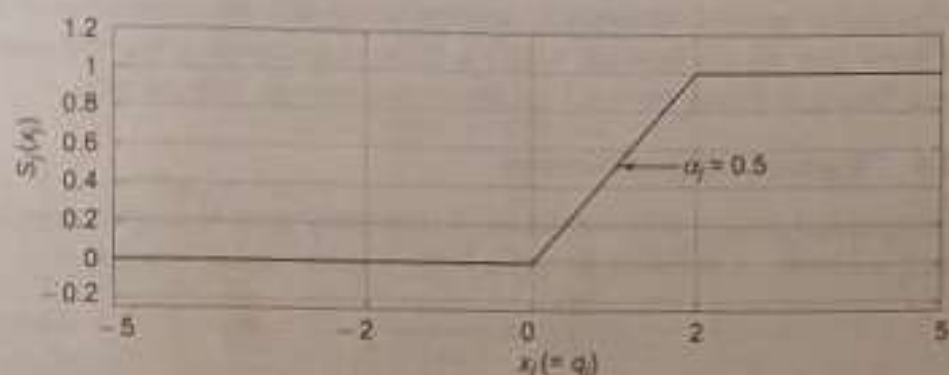
3.2.3 Linear Threshold (Ramp) Signal Function

The McCulloch-Pitts model for binary threshold neurons has been generalized in various ways. One obvious generalization is to use signal functions other than the threshold function. The simplest function is the linear function (Eq. 3.3) where we have

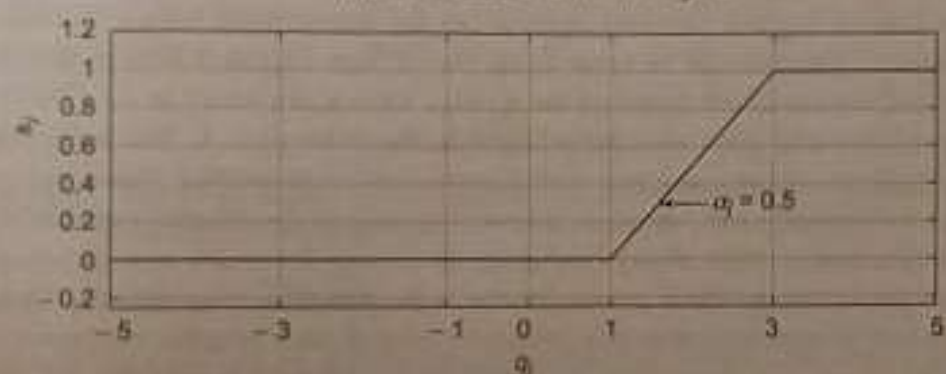
$$S_j(x_j) = x_j \quad (3.9)$$

This amounts to saying that the signal is equal to the activation untransformed in any way. However, such a signal function is unbounded. The bounded version of the linear signal function is the linear threshold function shown in Fig. 3.4.

We will encounter linear neurons in the context of maximal eigenvector filtering in Chapter 12, and the linear threshold signal function will be employed when we study the brain-state-in-a-box (BSB) model [22] in Section 10.13 and the Mexican Hat network in Section 12.7.



(a) Linear Threshold function: $\theta_j = 0$



(b) Shifted Linear Threshold function: $\theta_j = -1$

Fig. 3.4 Linear threshold neuron signal function with different neuronal thresholds as indicated

For a piecewise linear threshold signal function (Fig. 3.4a),

$$S_j(x_j) = \begin{cases} 0 & x_j \leq 0 \\ \alpha_j x_j & 0 < x_j < x_m \\ 1 & x_j \geq x_m \end{cases} \quad (3.10)$$

where $\alpha_j = 1/x_m$ is the slope parameter of the function. In Fig. 3.4(a), $x_m = 2$ and $\alpha_j = 0.5$. As before, the signal function shifts along the q_j axis depending upon the threshold value. This is shown in Fig. 3.4(b) for $\theta_j = -1$. You might have noticed that the negative threshold caused a right-shift of the signal function on the q_j axis. The signal function of such neurons can also be written in a more compact (and less instructive!) form as given

in Eq. 3.11.

$$S_j(x_j) = \max(0, \min(\alpha_j x_j, 1)) \quad (3.11)$$

For the sake of completeness, notice the introduction of an extra subscript j in the signal function $S_j(x_j)$, making it specific to the j th neuron. This is to emphasize that it is possible that each neuron has its own individual signal function, by maintaining a distinct value of the slope parameter α_j . However, all the neural network models we consider in this text are *homogeneous* in the sense that all the neurons within a network compute using the same signal function. Hence the need for the extra subscript will not arise.

3.2.4 Sigmoidal Signal Function

The sigmoid function is by far the most frequently used signal function in neural networks. A common example is the logistic signal function:

$$S_j(x_j) = \frac{1}{1 + e^{-\lambda_j x_j}} \quad (3.12)$$

where λ_j is a gain scale factor (see Fig. 3.5a). In the limit, as $\lambda_j \rightarrow \infty$ the smooth logistic function approaches the non-smooth binary threshold function. This should be clear from Fig. 3.5(a). Figure 3.5(b) shows the shifted version of the signal on the q_j axis, once again for $\theta_j = -1$. The 0.5 signal crossing point has shifted right to the point $q_j = 1$. The sigmoidal signal function has some very useful mathematical properties. These include its monotonicity and continuity. Loosely speaking, monotonicity means that the function y either always increases or always decreases as x increases. Continuity means there are no breaks in the function, it is smooth. These intrinsic properties eventually lend networks of such neurons the power to approximate and generalize on functions by learning from data. We will explore these ideas in considerable detail in Chapters 6, 7 and 8.

In some neural network applications, the logistic function is replaced by the hyperbolic tangent form,

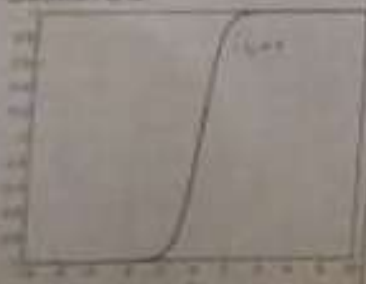
$$S_j(x_j) = k \tanh(\lambda_j x_j) \quad (3.13)$$

for some constant $k > 0$, that scales the signal range, and a slope factor, λ_j . The figure in the margin shows this signal function for $\lambda_j = 1$. The hyperbolic tangent signal function is an example of a *bipolar* signal function since the signal values can become both positive and negative, though continuously. Interestingly, one can generate a bipolar sigmoidal signal from the logistic signal function (see Problem 3.14).

The selection of the sigmoid as a neuron signal function is often justified by the biological argument that it represents the behaviour of the firing rate of action potentials, averaged over a population of neurons. In such a scenario, input and output signals represent ensemble averages [306].

Binary threshold neurons are limiting versions of sigmoidal neurons. Sigmoidal neuron networks will be the subject of our discussion in Chapter 6.

A more detailed discussion of these terms is forthcoming in Section 3.3.



Hyperbolic tangent signal function.

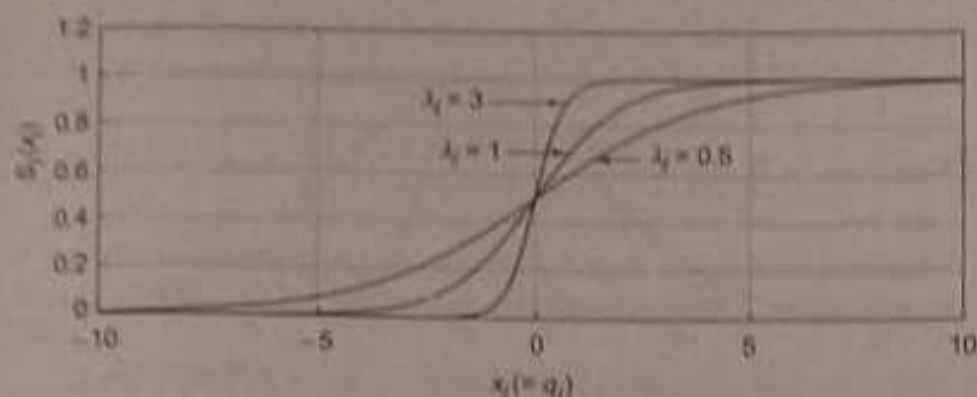
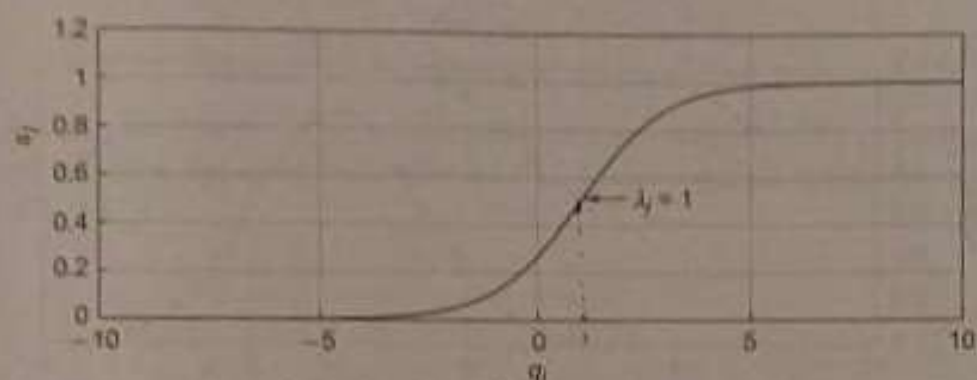
(a) Sigmoidal signal function $\theta_j = 0$ (b) Shifted sigmoidal signal function $\theta_j = -1$

Fig. 3.5 Sigmoidal signal function plotted for different gain scale factors and neuronal thresholds.

3.2.5 Gaussian Signal Function

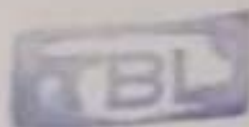
Another example of an interesting and powerful signal function is the Gaussian function:

$$S_j(x_j) = \exp\left(-\frac{(x_j - c_j)^2}{2\sigma_j^2}\right) \quad (3.14)$$

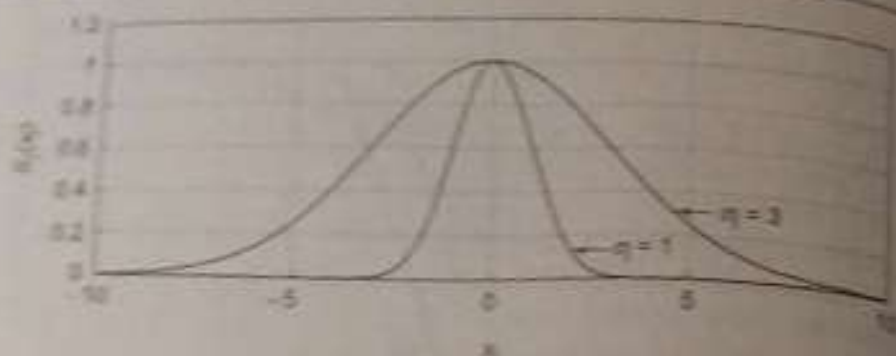
where σ_j is the Gaussian spread factor and c_j is the center. Such a signal function is portrayed in Fig. 3.6. Varying the spread makes the function sharper or more diffuse (Fig. 3.6a). Changing the center shifts the function to the right or left along the activation axis (Fig. 3.6(b)). This function is also an example of a *non-monotonic* signal function—it first increases from zero to its peak value of 1 at the centre and then decreases smoothly towards 0.

The use of Gaussian signal functions should be made intuitively clear. Activations close to the center of the function elicit strong signal responses. Those far away from the center elicit weak responses. There is thus a “receptive field” of the neuron within which it responds to inputs by generating strong signals. These neurons can therefore be used to *recognize* specific ranges of inputs by selectively tuning receptive fields to be centred around those regions of activity by using a suitable learning algorithm.

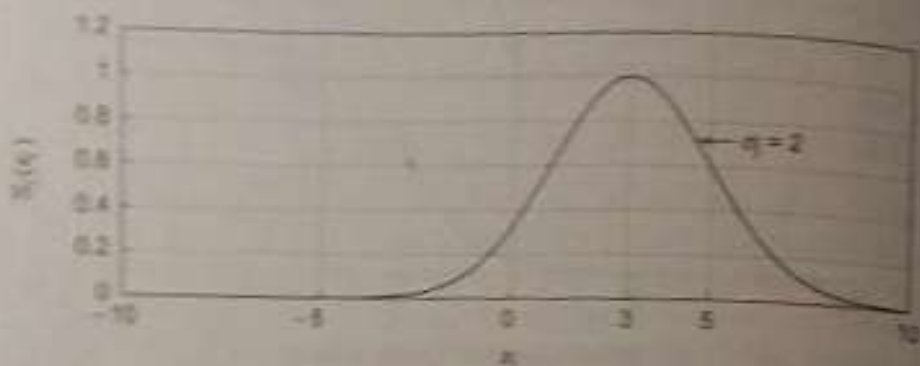
Radial basis function networks (RBFNs), employ non-monotonic Gaussian signal functions, and have a wide variety of applications. RBFNs will be discussed in detail in Chapter 8.



44817



(a) Gaussian signal function, center = 0



(b) Gaussian signal function, center = 3

Fig. 3.6 Gaussian signal function for various spreads

Like its sigmoidal counterpart, the Gaussian function takes as input an infinite range of activations $(-\infty, \infty)$, and transforms them smoothly into a value in the finite range $(0, 1)$.

3.2.6 Stochastic Neurons

Notice that all the neuron signal functions introduced thus far are deterministic in the sense that the signal value is completely determined by the instantaneous activation value that the neuron acquires, in conjunction with the neuron signal function. There is no randomness. However, synaptic transmission in biological neurons is essentially a noisy process brought about through random fluctuations resulting from the release of neurotransmitters, and numerous other uncertain causes. We wish to account for such randomness generated through synaptic noise in a mathematically tractable framework.

An easy way to introduce stochasticity into the neuron is to assume that its activation-to-signal update is no longer deterministic. For binary or bipolar stochastic neurons, the signal is assumed to be two state $s_j \in \{0, 1\}$ or $\{-1, 1\}$, but the neuron switches into these states depending upon a

probabilistic function of its activation, $P(x_j)$. In other words, the signal state s_j of neuron j switches in accordance with a probabilistic rule:

$$s_j = \begin{cases} +1 & \text{with probability } P(x_j) \\ -1 & \text{with probability } 1 - P(x_j) \end{cases} \quad (3.15)$$

If the neuron activation x_j is zero, we set $s_j \pm 1$ each with probability $\frac{1}{2}$. This stochastic rule is based on the Glauber dynamics model of Ising spins [170].

For the function $P(x_j)$ to qualify as a probability, it must satisfy the limiting values given in Eq. 3.16.

$$P(x_j) = \begin{cases} 0 & \text{as } x_j \rightarrow -\infty \\ 1 & \text{as } x_j \rightarrow +\infty \end{cases} \quad (3.16)$$

Further, the function $P(x_j)$ must increase monotonically with x_j between these limits. The common choice for $P(x_j)$ is the sigmoid-shaped function (Eq. 3.17):

$$P(x_j) = \frac{1}{1 + e^{-x_j/T}} \quad (3.17)$$

where T is a *pseudo-temperature* that controls the noise level (and therefore the uncertainty in the firing of a neuron). It is a parameter that controls the (thermal!) fluctuations in signal state that represent the effects of synaptic noise. For brevity, we refer to T in the context of neural networks as simply "temperature" hereafter. In the noiseless limit when $T \rightarrow 0$, the function transitions to a step, and the probabilistic firing rule of Eq. 3.15 becomes a purely deterministic rule of Eq. 3.5. The probability function $P(x_j)$ is plotted in Fig. 3.7 for various temperatures T .

How do we simulate such a signal function? Simulation of such a probabilistic function simply requires the generation of a random number drawn from a uniform distribution in the interval $[0, 1)$. If the number generated is less than or equal to $P(x_j)$ then the neuron switches to the +1 state; otherwise it switches to the -1 state. If $T = 0$, $P(x_j \geq 0) = 1$, and $P(x_j < 0) = 0$. In such a case a neuron switches deterministically, and the signal function reverts to the binary threshold case.

The interpretation of the probability function of Fig. 3.7 is important. The function represents the probability that the neuron switches to a +1 state—it may or it may not switch. Do not confuse this rule with the deterministic sigmoidal signal function of Eq. 3.12.

The quantum mechanical term *spin* refers to atomic magnets that comprise a description of a magnetic material. The atomic magnets are arranged onto a lattice, with their spins pointing in one of many possible directions. In an Ising spin model, the spin can be oriented either upwards or downwards—it can be +1 or -1—something that is called a spin 1/2 atom.

Note that T is not the physical temperature of a biological or artificial neural network!

Stochastic neurons are used in reinforcement learning networks reviewed in Chapter 6, and the Boltzmann machine which we will study in Chapter 10.

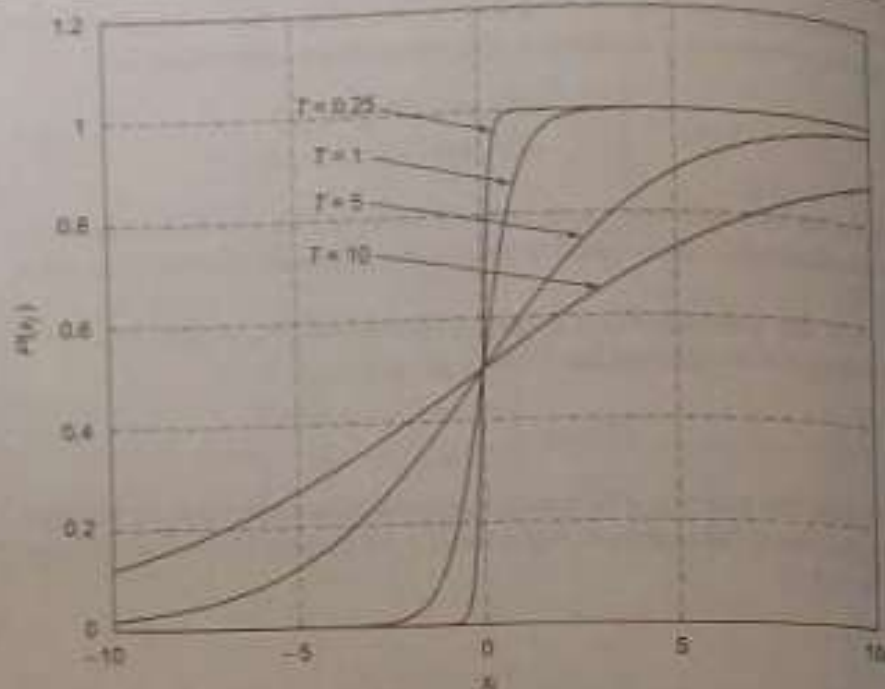
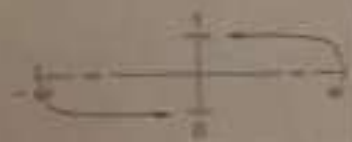


Fig. 3.7 Sigmoidal probability function plotted at various temperatures

3.2.7 Summary of Signal Functions



Signal functions squash the real line into a finite interval—an unbounded interval into a bounded one.

All the signal functions we have visited over the past few sections compress or *squash* an infinite range of activations into a finite signal range $[0, 1]$ or $[0, 1]$. These signal functions are therefore often referred to as *squashing* functions [323]. Table 3.1 summarizes what we have just learnt regarding various neuronal signal functions. Spend a few minutes on it before moving on.

You might have observed the obvious: all the artificial neuron models discussed above leave neurobiological concepts far behind. Only the neuron metaphor remains. In this context we might mention that there are indeed many more biologically realistic neuron models that have been introduced into the literature. These include:

- the spiking neuron model [363] where neuron activations are computed by weighted superposition of postsynaptic potentials and refractory functions.
- the integrate-and-fire model [306] which employs an electrical model to integrate postsynaptic potentials.
- conductance based models such as the Hodgkin-Huxley model and compartmental models, that are based on the workings of neurons at an ionic level [306].

Table 3.1 Summary of some common neuronal signal functions used in the neural network literature

Name	Function	Characteristics
Binary threshold	$S(x_j) = \begin{cases} 1 & x_j \geq 0 \\ 0 & x_j < 0 \end{cases}$	Non-differentiable, step-like, $x_j \in [0, 1]$
Bipolar threshold	$S(x_j) = \begin{cases} 1 & x_j \geq 0 \\ -1 & x_j < 0 \end{cases}$	Non-differentiable, step-like, $x_j \in [-1, 1]$
Linear	$S_j(x_j) = a_j x_j$	Differentiable, unbounded, $x_j \in (-\infty, \infty)$
Linear threshold	$S_j(x_j) = \begin{cases} 0 & x_j \leq 0 \\ a_j x_j & 0 < x_j < x_m \\ 1 & x_j \geq x_m \end{cases}$	Differentiable, piece-wise linear, $x_j \in [0, 1]$
Sigmoid	$S_j(x_j) = \frac{1}{1 + e^{-\lambda_j x_j}}$	Differentiable, monotonic, smooth, $x_j \in (0, 1)$
Hyperbolic tangent	$S_j(x_j) = \tanh(\lambda_j x_j)$	Differentiable, monotonic, smooth, $x_j \in (-1, 1)$
Gaussian	$e^{-(x_j - c_j)^2 / 2\sigma_j^2}$	Differentiable, non-monotonic, smooth, $x_j \in (0, 1)$
Stochastic	$S_j(x_j) = \begin{cases} +1 & \text{with probability } P(x_j) \\ -1 & \text{with probability } 1 - P(x_j) \end{cases}$	Non-deterministic step-like, $x_j \in (0, 1)$ or $[-1, 1]$

Note: The last six signal functions have been subscripted to emphasize that there are parameters (a_j , λ_j , σ_j , c_j) that make the neurons' signal function distinct.

We discuss these models in considerable detail in Chapter 13 where we also point out that certain computational constructs that are possible with these models have no counterpart in the artificial world.

Before we go ahead to formally define neural networks, we will stop by for a breezy review of important mathematical concepts that will be used in the text. Brace yourself!

3.3 Mathematical Preliminaries

In this section we introduce some notation, definitions and concepts that will be useful to understand the mathematical details of important neural network models. Any further details of math tools required will be introduced as we go along.

For a more detailed treatment of the various topics introduced in this section refer to the bibliographic remarks at the end of the chapter.

3.3.1 Notation

We consistently employ the following notation. Simple upper case alphabets will denote vectors whereas boldfaced upper case alphabets denote matrices. Therefore, X is a vector and A is a matrix. Vector or matrix components are lower case alphabets with appropriate subscripts indexing them. Therefore, a vector $X = (x_1, \dots, x_n)^T$ and matrix $A = [a_{ij}]$. Note the use of the transpose "T" for the vector since all vectors are assumed to be in columnar

form. As far as possible the symbols i, j, l will be employed to denote components of such vectors and matrices whereas k will be used to denote an iteration index or a discrete time index. Iteration or discrete time indices will be subscripted in vectors, and superscripted in their respective components. Therefore, the vector X_k at iteration k will have components $X_k = (x_1^k, \dots, x_n^k)^T$. The same goes for matrix components. A scalar quantity will have its iteration or discrete time index subscripted.

3.3.2 Intervals, Sequences, and Set Closure

In the sequel, we consistently assume the following notation— $\mathbb{R}$ denotes the set of real numbers; $\mathbb{R}^+$ denotes the positive reals; $\mathbb{Z}$ denotes the set of integers; and $\mathbb{Z}^+$ denotes the set of positive integers.

Intervals

Recall that an interval $I \subset \mathbb{R}$ is a set of real numbers that satisfy the property that if $x \in I$ and $y \in I$ and $x \leq z \leq y$, then $z \in I$. In describing intervals we usually employ the following notations as shown in Eqs 3.18 to 3.23.

$$(a, b) = \{x : a < x < b\} \quad (3.18)$$

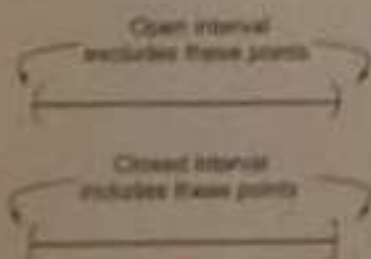
$$[a, b] = \{x : a \leq x \leq b\} \quad (3.19)$$

$$(a, \infty) = \{x : x > a\} \quad (3.20)$$

$$[a, \infty) = \{x : x \geq a\} \quad (3.21)$$

$$(-\infty, b) = \{x : x < b\} \quad (3.22)$$

$$(-\infty, b] = \{x : x \leq b\} \quad (3.23)$$



Note that these intervals are *bounded* either above or below or both. This simply means that there exists a real number h that is greater than every element of I , or a number l that is less than every number in I .

The *continuum* property of real numbers states that every non-empty set of real numbers which is bounded above has a least upper bound (*lub*); and which is bounded below has a greatest lower bound (*glb*). It follows therefore that each of the intervals considered above have a least upper bound or a greatest lower bound or both.

If $glb(I) \in I$ and $lub(I) \in I$, then I is *closed*. Otherwise I is either *half-open* or *open*. Also, $(a, \infty) = \{x : x \geq a\}$ and $(-\infty, b) = \{x : x \leq b\}$ are called *half-open rays*.

Sequences

A sequence $\{x_n\} \in I$ is a mapping from the set of integers, commonly $\mathbb{Z}^+$, into I . A point $x \in \mathbb{R}$ is a *limit point* of I if there is a sequence of points $\{x_n\} \in I$ that converges to x . For example, $\{\frac{1}{n}\}_{n=1}^{\infty}$ is a sequence of points in the interval $[0, 1]$ that converges to the point 0. Therefore, 0 is a limit point of the sequence $\{\frac{1}{n}\}_{n=1}^{\infty}$. Similarly the sequence $\{1 - \frac{1}{n}\}_{n=1}^{\infty}$ converges to the

point 1. Clearly, 0 and 1 are limit points of the open and closed intervals $(0, 1)$ and $[0, 1]$.

Set Closure

One can now define the closure of a set in terms of its *limit points*. A set together with its limit points is called its closure. A set $\mathcal{I}$ is closed if it contains all of its limit points. In the above example, $(0, 1)$ is therefore open and $[0, 1]$ is closed.

The complement of a closed set is open. By definition, an empty set is closed. Further, any finite union of closed sets is closed. But surprisingly infinite unions of closed sets need not be closed. For example if $\mathcal{I}_n = [\frac{1}{n}, 1]$, then $\mathcal{I} = \bigcup_{n=1}^{\infty} \mathcal{I}_n = (0, 1]$ is not closed since 0 is a limit point of $\mathcal{I}$ which is not in $\mathcal{I}$. On the other hand, infinite intersections of closed sets are closed. Unlike closed sets, infinite unions of open sets are open in $\mathbb{R}$. However, infinite intersections of open sets need not be open in $\mathbb{R}$. For example, if $\mathcal{J}_n = (-\frac{1}{n}, \frac{1}{n})$, then $\mathcal{J} = \bigcap_{n=1}^{\infty} \mathcal{J}_n = \{0\}$ (singleton 0) which is closed (by definition)!

Of particular importance are closed and bounded intervals which are *compact*.

We re-emphasize that in each of the signal functions studied in the previous section the range of activations can normally lie in the open interval $(-\infty, \infty)$, whereas the signals are often bounded in the closed intervals $[0, 1]$, $[-1, 1]$, $(0, 1)$, $(-1, 1)$, or limited to the set $\{0, 1\}$.

3.3.3 Vector Spaces

Vector spaces underlie the operation of almost all neural network models, and so it is of paramount importance to have the geometry clear. It is in the real n -dimensional vector space $\mathbb{R}^n$, that most neural network models find their operational home.

The Vector Space $\mathbb{R}^n$

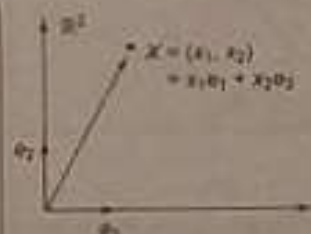
Vector spaces comprise a set of vectors of the form $X = (x_1, \dots, x_n)^T \in \mathbb{R}^n$. Here $x_1, x_2, \dots, x_n$ are real numbers, and are called the coordinates or components of X . We denote $\mathbb{R}^n$ as the set of real n -dimensional vectors, and call it the real n -dimensional *vector space*. Vector spaces comprise sets of vectors with operations of addition and multiplication defined on the set. Recall that vector spaces must also satisfy various important properties: closure of addition and multiplication, commutativity and associativity of addition; the existence of additive and multiplicative identity and inverses, and linearity. Of course, in the limiting case $\mathbb{R}^1 = \mathbb{R}$, the set of real numbers, which is a vector space in its own right.

A set of vectors $\{X_1, \dots, X_Q\}$, $X_i \in \mathbb{R}^n$ are said to be *linearly independent* if there exist Q scalars $\alpha_1, \dots, \alpha_Q$ such that

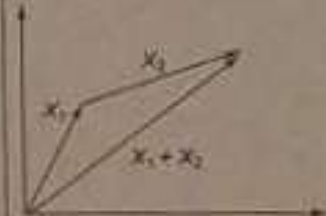
$$\alpha_1 X_1 + \dots + \alpha_Q X_Q = 0 \quad (3.24)$$

Equation 3.24 implies $\alpha_1 = \dots = \alpha_Q = 0$. What this really means is that one cannot generate the zero vector using any linear combination of the Q vectors.

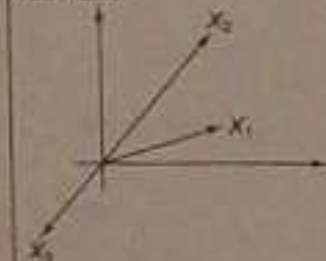
It is useful to look at vectors in $\mathbb{R}^n$ as being linear combinations of columns of the $n \times n$ identity matrix $I = [e_1 | e_2 | \dots | e_n]$. The coordi-



A vector represents a point in space.



Tail to head addition yields the resultant.



$\{X_1, X_2\}$ are linearly independent; $\{X_1, X_2, X_3\}$ are linearly dependent.

entries of a vector X are precisely the scalars that must multiply the columns of I to generate X :

$$X = x_1 e_1 + x_2 e_2 + \cdots + x_n e_n \quad (3.25)$$

In fact any vector in $\mathbb{R}^n$ can be generated in this way. We call the set of vectors $\{e_i\}_{i=1}^n$ the *elementary basis* of $\mathbb{R}^n$ and say that they *span* the vector space $\mathbb{R}^n$. Formally, any set of n linearly independent vectors in $\mathbb{R}^n$ span the vector space and form the basis, n being the dimension of the space. You can imagine that changing the basis changes the coordinates of the same physical points in the same space. We are of course used to the coordinate system being based on the elementary basis. A change of basis has a particularly important application in the case of linear maps, where it can help to diagonalize matrix operators.

Inner Product, Norm and Orthogonality

Of considerable importance in neural network theory is the scalar *inner product* (X, Y) of two vectors X and Y . In general, inner products satisfy three properties: $(X, Y) = (Y, X)$; $(X, a_1 Y_1 + a_2 Y_2) = a_1 (X, Y_1) + a_2 (X, Y_2)$, and $(X, X) \geq 0$ where $(X, X) = 0$ iff $X = 0$. Recall that "iff" denotes if and only if. The commonly employed inner product in $\mathbb{R}^n$ is:

$$(X, Y) = X^T Y = \sum_{i=1}^n x_i y_i \quad (3.26)$$

The norm is also a scalar function defined on a space of vectors that is based on the concept of vector magnitude. The norm $\|X\|$ of a vector $X \in \mathbb{R}^n$ satisfies three properties: $\|X\| \geq 0$ where $\|X\| = 0$ iff $X = 0$; $\|\alpha X\| = |\alpha| \|X\|$ for some scalar α ; and $\|X + Y\| \leq \|X\| + \|Y\|$. There are many norms that can be defined, and we shall see some of these in chapters to come. We are familiar with the standard Euclidean l^2 norm which is based on an inner product:

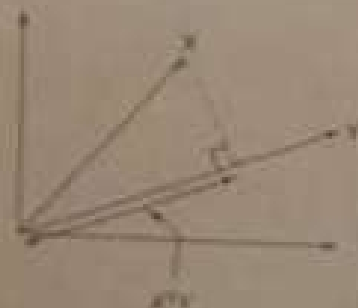
$$\|X\| = \sqrt{X^T X} = \sqrt{x_1^2 + \cdots + x_n^2} = \left(\sum_{i=1}^n x_i^2 \right)^{1/2} \quad (3.27)$$

These definitions permit us to write the inner product of two vectors using their norms and the angle γ they make with each other:

$$X^T Y = \|X\| \|Y\| \cos \gamma \quad (3.28)$$

The inner product is also called the projection of one vector on the other. Inner products measure similarity as we have already seen. Also, two vectors are *orthogonal* if they satisfy the condition $X^T Y = 0$ and in general if $(X, Y) = 0$. Intuitively, the product of the square of the magnitudes of two vectors can never be less than the square of their inner product. This is stated in the form of the *Cauchy-Schwarz inequality*:

$$(X^T Y)^2 \leq \|X\|^2 \|Y\|^2 \quad (3.29)$$



The inner product is the projection of one vector on the other.

3.3.4 Functions and Continuity

The importance of signal function *monotonicity* and *continuity* can hardly be overemphasized in the context of neural networks. These two properties are critical to the derivation of important global properties of neural networks and learning algorithms which we study in the chapters to come.

Continuous Functions

Recall that a *function* is a unique mapping from a domain space X into a range space Y written out as $f: X \rightarrow Y$. A function f is continuous at a point ζ if and only if $f(x) \rightarrow f(\zeta)$ as $x \rightarrow \zeta$. Geometrically this means that the graph of the function does not have a "break" or discontinuity in it. From calculus we know that a function is continuous at any point if the limit at that point exists and is equal to the value of the function, evaluated at the point in question. A function is continuous on an open interval I only if it is continuous at every point of I . When the interval $I = [a, b]$ then the function is continuous only if it is continuous at each point of I and continuous to the right at a and to the left at b . Continuous functions do not have any discontinuities. For example, the binary threshold signal function of Fig. 3.2(a) is discontinuous, whereas the sigmoidal function of Fig. 3.5(a) is continuous.

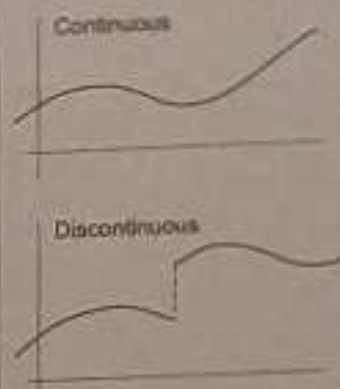
For simplicity of exposition we consider the one dimensional function $f: \mathbb{R} \rightarrow \mathbb{R}$. We denote the derivative of f at x by $f'(x)$. Higher derivatives are denoted by $f^{(i)}(x)$. We say that f is of class C^r on an interval I if $f^{(i)}(x)$ exists and is continuous at all $x \in I$. A function is *smooth* if it is C^1 .

Homeomorphisms

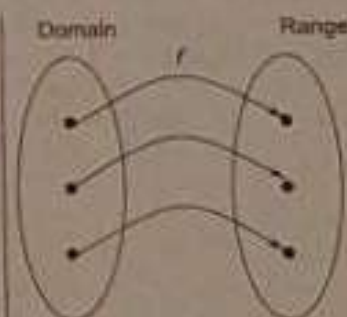
In general a function f is a *one-one* (or *injective*) function if every element in the domain maps to exactly one element in the range. Alternatively, for a one-one map, we say that $f(x) = f(y) \implies x = y$. A function $f: I \rightarrow J$ defined on compact intervals I, J is *onto* (or *surjective*) if for any $y \in J$ there exists $x \in I$ such that $f(x) = y$. One-one onto functions are important because they are invertible, and we define the inverse of f as $f^{-1}(x)$ in accordance with the rule that $f^{-1}(y) = x$ iff $f(x) = y$. Finally, the function $f(x)$ is a *homeomorphism* if $f(x)$ is one-one, onto, continuous, and $f^{-1}(x)$ is also continuous.

Monotonic Functions

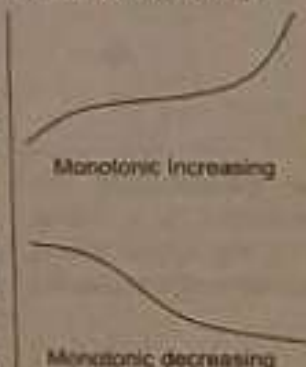
A *monotone* function of a real variable x is always either an increasing function of x , or always a decreasing function of x . In other words the slope maintains the same sign on the domain over which the function is defined. Monotonic functions uniformly increase or decrease. They have no "bumps". Accordingly they can be either *monotone-increasing* ($f'(\cdot) > 0$) or *monotone-decreasing* ($f'(\cdot) < 0$). The sigmoidal signal function of Fig. 3.5(a) is an example of a monotone increasing function. In contrast, the Gaussian signal function of Fig. 3.6(a) has a "bump". The



A continuous function is smooth, a discontinuous one breaks suddenly.



A one-one onto map.



slope changes its sign (at $x_j = 0$) from positive to negative and therefore the function is non-monotonic.

3.3.5 Matrices, Linear Transformations, and Eigenvectors

Matrices play a crucial role in neural network theory, and so it is best to spend a few moments on understanding some of their more important properties. We start by recalling that a matrix is a rectangular array of real or complex numbers, and in the present context we are interested only in matrices with real valued entries. Matrices that have an equal number of rows and columns are called square matrices.

Matrix Multiplication

A matrix can (and should) be looked upon as a horizontal stack of column vectors, or a vertical stack of row vectors. This point of view is especially useful when we multiply a row matrix into a rectangular matrix for it allows us to view the end result as a linear combination of the rows:

$$(\alpha \ \beta \ \gamma) \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} = \alpha (a_{11} \ a_{12}) + \beta (a_{21} \ a_{22}) + \gamma (a_{31} \ a_{32}) \quad (3.30)$$

or as a linear combination of the columns:

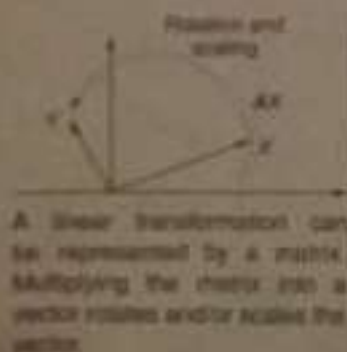
$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \alpha \begin{pmatrix} a_{11} \\ a_{21} \\ a_{31} \end{pmatrix} + \beta \begin{pmatrix} a_{12} \\ a_{22} \\ a_{32} \end{pmatrix} \quad (3.31)$$

A common operation performed on matrices is the transpose operation. Consider a matrix A . Its transpose, denoted by $A^T (= B)$ has its elements defined by $b_{ij} = a_{ji}$. Therefore, rows transpose to columns and vice-versa. Recall that matrix multiplication is not commutative, that is $AB \neq BA$, and in fact $(AB)^T = B^T A^T$. It is always useful to visualize two special cases of matrix multiplication: a row vector into a column vector—which we know is called the *inner product*—results in a scalar; and a column vector into a row vector—called the *outer product* results in a matrix whose dimensions depend on those of the row and column vectors.

Linear Transformation

A linear transformation A (or just a linear map) defined in a space such as $\mathbb{R}^n$ satisfies two important properties: $A(X_1 + X_2) = A(X_1) + A(X_2)$, $\forall X_1, X_2 \in \mathbb{R}^n$, and $A(\alpha X) = \alpha A(X) \forall X \in \mathbb{R}^n, \alpha \in \mathbb{R}$. Linear transformations generally scale and rotate points in $\mathbb{R}^n$.

To every linear map A between two finite dimensional vector spaces, a matrix A can be associated. This can be shown easily by choosing two vectors drawn from possibly different vector spaces, then expanding them in terms of their basis vectors, and finally invoking the properties of linear transformations. Matrix multiplication is an example of a linear map. It is



worth mentioning that there are some transformations that will only rotate points without scaling them. Recall the matrix $\begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$ which rotates vectors in $\mathbb{R}^2$ without scaling.

Eigenvalues and Eigenvectors

An extremely important concept that recurs in our treatment of neural networks is the idea of an eigenvector and its associated eigenvalue. We first define them both and then provide intuitive feel for the concept.

Definition 3.3.1

A non-zero vector η_i is called an eigenvector of A if it satisfies the condition given in Eq. 3.32,

$$A\eta_i = \lambda_i \eta_i \quad (3.32)$$

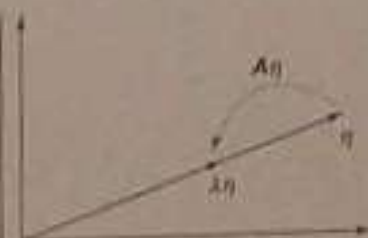
for some real scalar λ_i , called the eigenvalue. We usually say that η_i belongs to λ_i .

It is extremely useful to think of eigenvectors η_i as vectors that are direction invariant under the mapping represented by matrix A . This observation follows directly from the definition of an eigenvector. Notice that matrix A operating on point η_i in $\mathbb{R}^n$ maps it to a new point $\lambda_i \eta_i$ which is simply the same vector η_i scaled by a real scalar λ_i . Therefore, the image of η_i retains the direction of η_i under the linear map A represented by matrix A , although its magnitude changes by a scale factor λ_i . We know that in the general case, a linear map rotates as well as scales points in space. However, the above observations mean that for each such map, there are at most n directions in $\mathbb{R}^n$, such that all points along those directions retain their direction under the map. These directions are precisely the eigenvector directions (or eigendirections).

To find the eigenvalues one needs to closely follow Eq. 3.32. In general, for any eigenvector η belonging to an eigenvalue λ , it follows from Eq. 3.32 that

$$(A - \lambda I)\eta = 0 \quad (3.33)$$

where $A - \lambda I : \mathbb{R}^n \rightarrow \mathbb{R}^n$ represents a map that takes points $\eta \in \mathbb{R}^n$ to 0. From Eq. (3.33) it follows that eigenvectors are vectors that lie in the kernel of the map represented by $A - \lambda I$. However, since an eigenvector is a non-zero vector by definition ($\eta_i \neq 0$), the map represented by $A - \lambda I : \mathbb{R}^n \rightarrow \mathbb{R}^n$ must have a non-trivial kernel. Since the kernel is non-zero, the map cannot be 1-1 (see explanation in marginal note). Now a square matrix is invertible only if it represents a 1-1 map. We therefore conclude that the matrix $A - \lambda I$ is not invertible. It is singular with determinant zero.



Multiplying a matrix into its eigenvector does not rotate the vector, it only scales by the eigenvalue.

The kernel of a linear map A is the set of points that map to 0. $\text{Ker } A = \{X \in \mathbb{R}^n | A(X) = 0\} = A^{-1}(0)$. A linear map A is 1-1 if and only if $\text{Ker } A = 0$. For suppose that $X \in \text{Ker } A$. Then $A(X) = 0 = A(0)$. Hence $X = 0$, and 0 is the only element of $\text{Ker } A$. Conversely, if $\text{Ker } A = 0$ and $A(X) = A(Y)$. Then $A(X - Y) = 0$, so $X - Y \in \text{Ker } A$. Thus $X - Y = 0$ so $X = Y$.

Consequently,

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0 \quad (3.34)$$

Here, we have assumed that λ is real, although λ could as well be complex. See [237] for a detailed treatment of this case.

Equation 3.34 is called the *characteristic polynomial* of the matrix $\mathbf{A}$ and solutions of this equation are called *eigenvalues* of matrix $\mathbf{A}$.

3.3.6 Expectations and Covariance Matrices

Often, we will be concerned with certain statistical properties of the input data stream. In particular, we will be dealing with expectations and covariance matrices.

The expectation of the vector $\mathbf{X} = (x_1, \dots, x_n)^T$ is a vector $\bar{\mathbf{X}}$ whose elements $(\bar{x}_1, \dots, \bar{x}_n)$ are given by Eq. 3.35,

$$\bar{x}_i \triangleq \int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty} x_i p(\mathbf{X}) dx_1 \dots dx_n \quad (3.35)$$

where $p(\mathbf{X})$ is the probability density function that governs the distribution. We use the notation $E[\mathbf{X}]$ for the expectation of $\mathbf{X}$.

The covariance matrix $\mathbf{K}$ associated with real random vector $\mathbf{X}$ is the expected value of the outer vector product $(\mathbf{X} - \bar{\mathbf{X}})(\mathbf{X} - \bar{\mathbf{X}})^T$,

$$\mathbf{K} \triangleq E[(\mathbf{X} - \bar{\mathbf{X}})(\mathbf{X} - \bar{\mathbf{X}})^T] \quad (3.36)$$

where individual elements σ_{ij} of $\mathbf{K}$ are defined as follows:

$$\sigma_{ij} \triangleq E[(x_i - \bar{x}_i)(x_j - \bar{x}_j)] \quad (3.37)$$

$$= E[(x_j - \bar{x}_j)(x_i - \bar{x}_i)] \quad (3.38)$$

$$= \sigma_{ji} \quad i, j = 1, \dots, n \quad (3.39)$$

Note that in an expanded form, $\mathbf{K}$ looks like the following matrix (Eq. 3.40).

$$\mathbf{K} = \begin{bmatrix} \sigma_{11}^2 & \dots & \sigma_{1n} \\ \vdots & \ddots & \vdots \\ \sigma_{n1} & \dots & \sigma_{nn}^2 \end{bmatrix} \quad (3.40)$$

When $\mathbf{X}$ is real, the elements of $\mathbf{K}$ are real, and also symmetric since $\sigma_{ij} = \sigma_{ji}$. Covariance matrices are *real symmetric* matrices. Such matrices fall within the larger class of *Hermitian* matrices—the class of $n \times n$ matrices for which $\sigma_{ij} = \sigma_{ji}^*$, where the $*$ is the complex conjugate operation. When $\mathbf{X}$ is complex, the covariance matrix is generally Hermitian. Real symmetric matrices have many interesting properties which we will not delve into here. See [230] for more on this.

You may have noticed that the diagonal elements σ_{ii}^2 are the variances associated with the individual random variables x_i for $i = 1, \dots, n$. The covariance matrix $\mathbf{K}$ is closely related to the correlation matrix $\mathbf{R}$, defined by Eq. 3.41.

$$\mathbf{R} \triangleq E[\mathbf{X}\mathbf{X}^T] \quad (3.41)$$

Indeed expanding Eq. 3.36 yields

$$\mathbf{K} = \mathbf{R} - \bar{\mathbf{X}}\bar{\mathbf{X}}^T \quad (3.42)$$

or

$$\mathbf{R} = \mathbf{K} + \bar{\mathbf{X}}\bar{\mathbf{X}}^T \quad (3.43)$$

The correlation matrix $\mathbf{R}$ is also symmetric; it is sometimes called the auto-correlation or scatter matrix.

Done with this review, we can now turn our attention back to what neural networks are, and what are their important properties.

3.4 Neural Networks Defined

Ensembles of interconnected artificial neurons generally organized into layers or *fields* comprise neural networks. However, the behaviour of such ensembles varies greatly with changes in architecture as well as the neuron signal functions. We adopt a working definition of an artificial neural network from Kohonen's original contribution [312].

Definition 3.4.1

Artificial neural networks are massively parallel adaptive networks of simple nonlinear computing elements called neurons which are intended to abstract and model some of the functionality of the human nervous system in an attempt to partially capture some of its computational strengths.

An important point to note is that the fundamental computing elements retain their computational simplicity, and at a global level, massively parallel interactions can lead to complex and interesting behaviour that can be put to use in solving different problems. The study of neural networks focusses both on the analysis and design of such massively parallel computing systems.

In its most general form, a neural network can be viewed as comprising eight components as summarized below [573].

- ⇒ **Neurons.** These can be of three types: input, hidden, and output. Input neurons are designated to receive external stimuli presented to the network. Outputs from the network are generated as signals of output neurons. Hidden neurons compute intermediate functions and their states are not accessible to the external environment.
- ⇒ **Activation state vector.** This is a vector of the activation level x_i of individual neurons in the neural network, $\mathbf{X} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$.
- ⇒ **Signal function.** A function that generates the output signal of the neuron based on its activation. Common functions are binary

threshold, linear threshold and sigmoidal functions. Functions may differ from neuron to neuron within the network, although most networks are *field-homogeneous* in the sense that all neurons within a field (or layer) have the same signal function.

- *Pattern of connectivity.* This essentially determines the inter-neuron connection architecture or the graph of the network. Connections which model the inter-neuron synaptic efficacies, can be excitatory (+) or inhibitory (-) or absent (0). The memory of the network resides in the connections, and it is the connections together with the neuron signal functions that determine the global behavioural properties of the network and thus the function it performs.
- *Activity aggregation rule.* This is the way of aggregating activity at a neuron, and is usually computed as an inner product of the input vector and the neuron fan-in weight vector (which in its augmented form includes the threshold).
- *Activation rule.* This is a function that determines the new activation level of a neuron on the basis of its current activation and its external inputs. In general, most neural network models ignore this intermediate function and assume that the new activation level is equal to the net activity aggregate computed above.
- *Learning rule.* The distinguishing property of neural networks lies in their ability to learn from examples. The learning rule provides a means of modifying connection strengths based both on external stimuli and network performance with an aim to improve the latter.
- *Environment.* The environments within which neural networks can operate could be deterministic (noiseless) or stochastic (noisy).

For models that do not ignore this component see Vol. 3 of the PDP Series [375].

3.5 Architectures: Feedforward and Feedback

A neural network N can be viewed as a weighted directed graph in which artificial neurons are nodes and directed weighted edges represent connections between neurons. Local groups of neurons can be connected in either:

- a *feedforward* architecture, in which the network has no loops, or
- a *feedback* (recurrent) architecture, in which loops occur in the network because of feedback connections.

It is worth noting that the entire computation in neural networks takes place through local interaction of neurons. Ensembles of such locally computing neurons yield specific global properties depending upon the network architecture and connection strengths. Different network architectures yield different behavioural patterns of varying complexity.

3.5.1 Neural Networks Generate Mappings

As already mentioned, the neurons in an artificial neural network are usually organized into fields or layers. Inputs to the network are presented to the input layer; outputs are generated as signals of the output layer. Since the numeric activations and signals comprise vectors we say that an input vector applied to the input layer generates an output signal vector across the output layer. These signals may pass through one or more intermediate or *hidden* layers which transform the signals depending upon the neuron signal functions. Look upon the network as a transformation—linear or non-linear—from the input space to the output space. For example if a 5-neuron input field feeds signals to a 4-neuron output field, vector inputs from a 5-dimensional space are transformed or mapped to signal vectors in a 4-dimensional space.

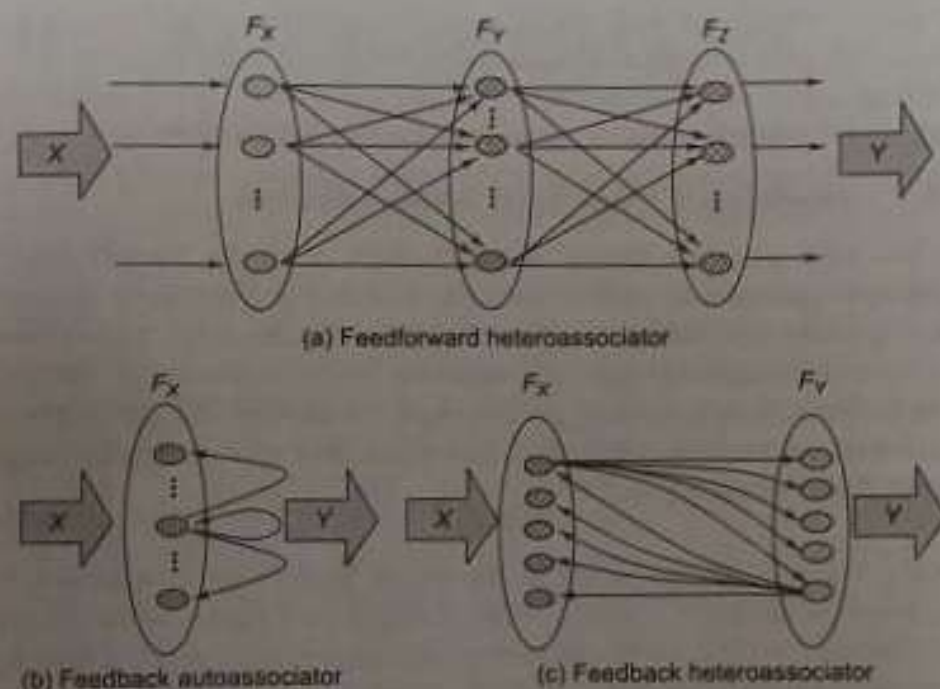


Fig. 3.8 Common neural network architectures

Neural networks thus generate mappings—usually from one space to another or sometimes from the input space back to itself. Figure 3.8 depicts these two patterns of connectivity in the form of three architectures. Multilayered networks that associate vectors from one space to vectors of another space are called *heteroassociators*. At any instant of time, they might map or associate two different patterns with one another—one as input and the other as output. Mathematically we write, $f: \mathbb{R}^n \rightarrow \mathbb{R}^p$. The neural network N takes a vector in $\mathbb{R}^n$ and maps it to a point in $\mathbb{R}^p$. When neurons in a single field connect back onto themselves the resulting network is called an *autoassociator* since it associates a single pattern in $\mathbb{R}^n$ with itself. In such a case, the neural network implements a

mapping $f: \mathbb{R}^n \rightarrow \mathbb{R}^p$. Figures 3.8(a) and 3.8(c) portray a feedforward and feedback heteroassociator respectively; Fig. 3.8(b) shows the architecture of a feedback autoassociator.

3.5.2 Activation and Signal State Spaces

Note that for a p -dimensional field of neurons, the activation state space is the entire $\mathbb{R}^p$. However, the signal state space is the Cartesian cross space,

$$\mathbb{I}^p = \underbrace{[0, 1] \times \cdots \times [0, 1]}_{p \text{ times}} = [0, 1]^p \subset \mathbb{R}^p \quad (3.44)$$

if the neurons have continuous signal functions in the interval $[0, 1]$, or $[-1, 1]^p$ if the neurons have continuous signal functions in the interval $[-1, 1]$. For the case when the neuron signal functions are binary threshold, the signal state space is

$$\mathbb{B}^p = \underbrace{\{0, 1\} \times \cdots \times \{0, 1\}}_{p \text{ times}} = \{0, 1\}^p \subset \mathbb{I}^p \subset \mathbb{R}^p \quad (3.45)$$

or $\{-1, 1\}^p$ when the neuron signal functions are bipolar threshold.

3.5.3 Feedforward vs Feedback: Static vs Dynamic

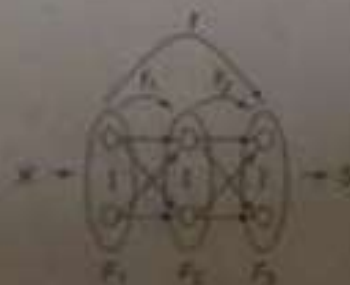
In the most common family of feedforward networks, loosely called *multilayer perceptrons* (which we study in detail in Chapter 6), neurons are organized into different layers that have unidirectional connections between them. Feedforward networks are static in the sense that the output depends only on the present input, in exactly the same way a combinational logic circuit produces an output for a specific Boolean input vector. Mathematically,

$$S = f(X) \quad (3.46)$$

where $S = (s_1, \dots, s_p)^T \in \mathbb{R}^p$ is the output signal vector generated in response to an input vector $X \in \mathbb{R}^n$. Feedforward neural networks are memory-less in the sense that their response to an input is independent of the previous network state. Although such networks possess no dynamics they are capable of demonstrating powerful properties such as universal function approximation and find widespread applications in pattern classification. In Chapters 4 through 8 we will clarify these properties further.

On the other hand, recurrent or feedback networks, are classical examples of non-linear dynamical systems. When a new input pattern is presented, the neuron outputs are computed as usual, but because these outputs are fed back as inputs to the system, the activations of neurons get subsequently modified, leading the network to a new state. The next state of the network is thus a function of the current input and the present state of the network, very much like a finite state machine. Feedback networks thus possess a rich repertoire of dynamics which result in their being capable of performing powerful tasks such as pattern completion [247], topological feature mapping [315] and pattern recognition [477].

The mapping into $\mathbb{I}^p$ is more general than $\mathbb{I}^p$ or $\mathbb{B}^p$ since these are contained within $\mathbb{I}^p$.



In a multilayered network the map from input to output space is a composition of sub-maps which are created between the layers.

In recurrent neural networks, network activations and signals are in a flux of change until (and unless) they settle down to a steady state. These changing quantities are readily modelled by first order nonlinear differential equations of the form:

$$\dot{x}_i = f_i(X) \quad i = 1, \dots, n \quad (3.47)$$

where $X = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ is a vector of activations of neurons in the network. As before, each neuron instantaneously generates an output signal (assuming a deterministic signal function) as a function of the activation. A trajectory in the state space $\mathbb{R}^n$ then describes the time evolution of network activations. Neural networks of the form just described represent complex nonlinear dynamical systems.

One of the most important issues that accompanies such models of nonlinear dynamical systems is the issue of stability. Given a feedback neural network architecture we must ensure that the network dynamics leads to a behaviour that can be interpreted in a sensible way. Dynamical systems have many variants of behaviour ranging from fixed point equilibria where the network eventually converges to a well defined point in the state space, to chaotic dynamics where the system wanders seemingly aimless in the state space [537]. We explore these aspects of dynamical systems in Chapter 9.

Ensuring the stability of a non-linear dynamical system is no simple task. Fortunately, in the neural network literature there are some well established results such as the *Cohen-Grossberg Theorem* [106] that guarantee the stability of a restricted class of neural networks. Here, stability is established on the basis of a rigorous mathematical analysis of *Lyapunov functions*. A number of neural network models are based on this theorem and in Chapters 10 and 11 we will discuss these in detail.

In general, it is not necessary that fields of neurons represent any specific *topological grouping*—although in some neural network models such as the Kohonen model, they do [314, 315]. In topological groupings, the topology of input stimuli is preserved and mapped onto the physical topology of the neural network. We shall have considerably more to say about topology preserving maps in Chapter 12.

On the basis of the above discussion, Table 3.2 provides a compact summary of various neural network models, their important characteristics and applications. We will come across these models during the course of our study.

In computer simulations we often use difference equations rather than differential equations.

In a topology preserving neural network map, input vectors that are close to one another (in terms of a distance measure) in the input space, are mapped to neurons (which compete with each other for maximum activation) in the network that are physically close to one another.

3.6 Salient Properties and Application Domains of Neural Networks

Before closing this chapter, we summarize some important properties of neural networks that have made them popular for real world applications

mapping $f: \mathbb{R}^k \rightarrow \mathbb{R}^k$. Figures 3.8(a) and 3.8(c) portray a feedforward and feedback heteroassociator respectively; Fig. 3.8(b) shows the architecture of a feedback autoassociator.

3.3.2 Activation and Signal State Spaces

Note that for a p -dimensional field of neurons, the activation state space is the entire $\mathbb{R}^p$. However, the signal state space is the Cartesian cross space,

$$\mathcal{P} = \underbrace{[0, 1] \times \cdots \times [0, 1]}_{p \text{ times}} = [0, 1]^p \subset \mathbb{R}^p \quad (3.44)$$

if the neurons have continuous signal functions in the interval $[0, 1]$, or $[-1, 1]^p$ if the neurons have continuous signal functions in the interval $[-1, 1]$. For the case when the neuron signal functions are binary threshold, the signal state space is

$$\mathcal{B}^p = \underbrace{[0, 1] \times \cdots \times [0, 1]}_{p \text{ times}} = [0, 1]^p \subset \mathcal{P} \subset \mathbb{R}^p \quad (3.45)$$

or $[-1, 1]^p$ when the neuron signal functions are bipolar threshold.

3.3.3 Feedforward vs Feedback: Static vs Dynamic

In the most common family of feedforward networks, loosely called *multilayer perceptrons* (which we study in detail in Chapter 6), neurons are organized into different layers that have unidirectional connections between them. Feedforward networks are static in the sense that the output depends only on the present input, in exactly the same way a combinational logic circuit produces an output for a specific Boolean input vector. Mathematically,

$$S = f(X) \quad (3.46)$$

where $S = (s_1, \dots, s_p)^T \in \mathbb{R}^p$ is the output signal vector generated in response to an input vector $X \in \mathbb{R}^k$. Feedforward neural networks are memory-less in the sense that their response to an input is independent of the previous network state. Although such networks possess no dynamics they are capable of demonstrating powerful properties such as universal function approximation and find widespread applications in pattern classification. In Chapters 4 through 8 we will clarify these properties further.

On the other hand, recurrent or feedback networks, are classical examples of *non-linear dynamical systems*. When a new input pattern is presented, the neuron outputs are computed as usual, but because these outputs are fed back as inputs to the system, the activations of neurons get subsequently modified, leading the network to a new state. The next state of the network is thus a function of the current input and the present state of the network, very much like a finite state machine. Feedback networks thus possess a rich repertoire of dynamics which results in their being capable of performing powerful tasks such as pattern completion [247], topological feature mapping [315], and pattern recognition [477].

The mapping into $\mathbb{R}^p$ is more general than $\mathcal{P}$ or $\mathcal{B}^p$ since these are contained within $\mathbb{R}^p$.

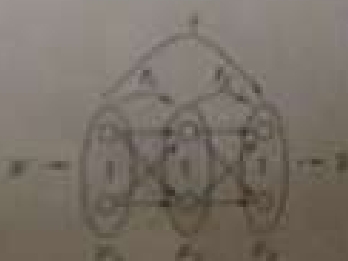


Fig. 3.8(a) A multilayered network. The map from input to output space is a composition of sub-maps which are created between the layers.

In recurrent neural networks, network activations and signals are in a flux of change until (and unless) they settle down to a steady state. These changing quantities are readily modelled by first order nonlinear differential equations of the form:

$$\dot{x}_i = f_i(X) \quad i = 1, \dots, n \quad (3.47)$$

where $X = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ is a vector of activations of neurons in the network. As before, each neuron instantaneously generates an output signal (assuming a deterministic signal function) as a function of the activation. A trajectory in the state space $\mathbb{R}^n$ then describes the time evolution of network activations. Neural networks of the form just described represent complex nonlinear dynamical systems.

One of the most important issues that accompanies such models of nonlinear dynamical systems is the issue of stability. Given a feedback neural network architecture we must ensure that the network dynamics leads to a behaviour that can be interpreted in a sensible way. Dynamical systems have many variants of behaviour ranging from fixed point equilibria where the network eventually converges to a well defined point in the state space, to chaotic dynamics where the system wanders seemingly aimless in the state space [537]. We explore these aspects of dynamical systems in Chapter 9.

Ensuring the stability of a non-linear dynamical system is no simple task. Fortunately, in the neural network literature there are some well established results such as the *Cohen-Grossberg Theorem* [106] that guarantee the stability of a restricted class of neural networks. Here, stability is established on the basis of a rigorous mathematical analysis of *Lyapunov functions*. A number of neural network models are based on this theorem and in Chapters 10 and 11 we will discuss these in detail.

In general, it is not necessary that fields of neurons represent any specific *topological grouping*—although in some neural network models such as the Kohonen model, they do [314, 315]. In topological groupings, the topology of input stimuli is preserved and mapped onto the physical topology of the neural network. We shall have considerably more to say about topology preserving maps in Chapter 12.

On the basis of the above discussion, Table 3.2 provides a compact summary of various neural network models, their important characteristics and applications. We will come across these models during the course of our study.

In computer simulations we often use difference equations rather than differential equations.

In a topology preserving neural network map, input vectors that are close to one another (in terms of a distance measure) in the input space, are mapped to neurons (which compete with each other for maximum activation) in the network that are physically close to one another.

3.6 Salient Properties and Application Domains of Neural Networks

Before closing this chapter, we summarize some important properties of neural networks that have made them popular for real world applications

Table 3.2 A summary of important neural network models, their characteristics, applications, and chapter numbers in which we study them

Model	Architecture Neuron Characteristics	Learning Algorithm	Application	Chapter
Perceptron	Single-node, feedforward Binary threshold	Supervised, error-correction	Pattern classification	3
Adaline	Single-node, feedforward Linear	Supervised, gradient descent	Regression	5
Multilayer perceptron	Multilayered, feedforward nonlinear sigmoid	Supervised, gradient descent	Function approximation	6
Reinforcement learning	Multilayered Binary threshold	Supervised reward-punishment	Control	8
Support vector machines	Multilayered kernel based, binary threshold	Supervised quadratic optimization	Classification, regression	8
Radial basis function net	Multilayered distance based, linear	Supervised gradient descent	Interpolation, regression, classification	8
Hopfield network	Single layer, feedback Binary threshold/sigmoid	Outer product correlation	CAM optimization	10
Boltzmann machine	Two layered, feedback Binary threshold	Stochastic gradient descent	Optimization	10
BSB	Single layered, feedback Linear threshold	Outer product correlation	Clustering	10
Bi-directional associative memory	Two layered, feedback Binary threshold	Outer product correlation	Associative memory	10
Adaptive resonance theory	Two layered Binary, faster than linear	Unsupervised competitive	Clustering, classification	11
Vector quantization	Single layer, feedback Faster than linear	Supervised, unsupervised competitive	Quantization, clustering	12
Mexican hat net	Single layer, feedback Linear threshold	None fixed weights	Activity clustering	12
Kohonen SOM	Single layer Linear threshold	Unsupervised self-competitive	Topological mapping	12
Polled neural models	Single/multilayer Perceptron/linear	None	Confidence detection Temporal processing	13

[269, 355, 356]. Amongst others, the primary ones are robustness, the capacity for associative recall, and the capability for function approximation and generalization.

3.6.1 Robustness

Neural networks continue to operate, albeit with degraded performance, even if some connections in the network get damaged. In fact, one of the main benefits claimed for neural networks is this *graceful degradation*. A

computational system is said to exhibit graceful degradation when it is able to continue to operate, albeit with some performance loss, in the event of damage to its internal structure. In other words, breaking connections in neural networks tends to gracefully degrade their performance. In contrast, symbolic AI systems do not degrade smoothly—a small corruption in the rule base can completely change the systems response, in much the same way as a digital VLSI system fails even if one connection is broken. Such systems are said to be brittle.

Neural networks are also robust in the sense that they can tolerate significant distortion in the inputs for which they are programmed. This property is called fault tolerance. The network performance continues to be satisfactory when inputs are distorted. The robustness of a neural network stems from the fact that it stores information in a distributed fashion—information about a single concept mapping or association is microscopically programmed into network connections. Consequently, as the example in the margin figure depicts, an occluded image can be reliably classified by a neural network.

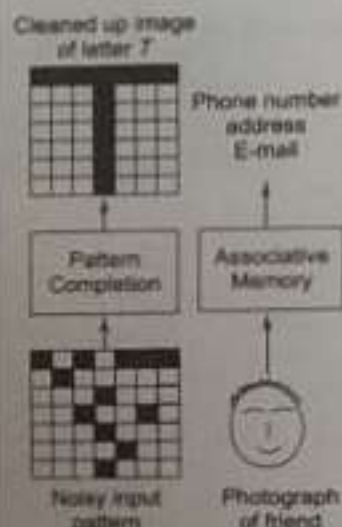


3.6.2 Associative Recall

Due to the dense inter-connectivity and reinforcing structure embedded in ensembles of connected neurons, *associative recall* becomes a natural capability for neural networks. In associative recall, one concept invokes related memories. A friend's name elicits vivid mental pictures and related emotions. However, distributed encoding causes *crosstalk* between concepts embedded in the connections of a neural network.

Such interference can become a major problem as new information is superimposed on information already learnt by the network. Conversely, older information distorts more recent information. Crosstalk thus limits the network storage capacity. The capacity is thus a function of the number of neurons and synapses in the network and is in a sense limited by the dimension of the network.

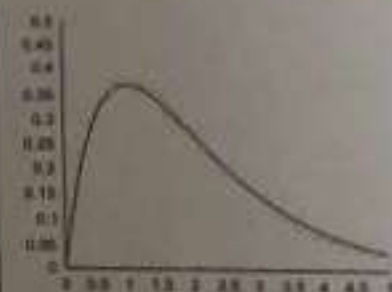
In the context of associative memory, some neural networks exhibit the *pattern completion* property. Here, an incomplete pattern of activation on a field of input units can lead to a known completion of the activity pattern appearing on the same or some other field.



Neural networks can clean up noisy images, or implement associative memory.

3.6.3 Function Approximation and Generalization

Neural networks are widely noted for *model free estimation* capability in the sense that they are able to create internal representations only through examples, without being supplied a mathematical model of how outputs depend upon inputs. Sometimes this is called *similarity based generalization*. Using powerful learning algorithms, they are able to approximate functions by sifting through vast repositories of data. Therefore, because of their learning capability, they are often referred to as *adaptive function estimators*.



Neural networks can approximate a function from a noisy and sparse data scatter.



Time series prediction is another major application



Neural networks find widespread application in control

In fact, neural networks are able to make generalizations based on previously learnt pattern class information. In this context neural networks also find application in time series prediction and control.

The inherent parallelism of neural networks makes them capable of seamlessly combining information from multiple sources which makes them sensitive to context. The conclusions drawn from the data depend on the context in which the information is presented. Due to this parallel nature of computation, it is possible to have hypotheses cooperate and compete with one another until a stable solution is found through network dynamics. This is called *soft constraint satisfaction*. We shall have ample opportunity of being exposed to each of these properties and applications as we study various networks in chapters to come.

Our study of the vast subject on neural networks starts with the geometry of the simplest of neurons: the threshold logic neuron. Chapter 4 focuses on the same.

Chapter Summary

- Artificial neurons are similar to their biological counterparts in the sense that they generate an internal activation based on a weighted summation of input signals. It should be clear that the quantum of processing that takes place in biological neurons is far more complex.
- The internal activation is transformed into a signal with the help of a signal function which could be any one—binary threshold, linear threshold, sigmoidal, Gaussian, or probabilistic. Bipolar versions of these signal functions are also used in contemporary neural network models. For example, the signum function and the hyperbolic tangent function have signal values that go both positive and negative. Note that this form of signalling is again vastly different from the action potential frequency based signalling encountered in biological neurons. More recently, pulse coded neural networks have been proposed in the literature (see the bibliographic remarks below). These networks employ artificial neurons that model the pulsed nature of action potentials in real neurons—they have a much greater biological fidelity.
- Probabilistic neurons update their signals based on a probability value derived from a sigmoidal probability curve that is a function of their activation. As the temperature is reduced, the sigmoid tends to the binary step function, and probabilistic update tends towards deterministic update.
- Neural networks connect artificial neurons in fundamentally two different architectures: feedforward and feedback. Feedforward networks are static in the sense that the output depends only on

the presently applied input. Feedback networks possess a rich repertoire of dynamics. Single layered networks are autoassociative; multilayered networks are heteroassociative.

- Depending on the kind of architecture, the neural network has an associated learning algorithm that allows it to adapt its internal parameters to improve its response to a desired input-output behaviour.
- Neural networks are well known for their
 - ▶ robustness, graceful degradation of performance, and fault tolerance.
 - ▶ ability to approximate functions.
 - ▶ capability to perform associative recall of memory and fill-in incomplete information.
 - ▶ automatic similarity based generalization.

The reader interested in going directly to the following references is welcome (and encouraged!) to do so. For a wonderful treatment of linear algebra there is no better place than Strang [536]. If you want to see linear algebra being put to direct use in dynamical systems, look no further than Hirsch and Smale [237]. An accessible and concise treatment of mathematical analysis is to be found in Binmore [55]. For a detailed treatment of the analysis of non-linear systems check Vidyasagar [566]. A wonderful treatment of dynamical systems is in Strogatz [537]. If one is interested in a more mathematical treatment of dynamical systems, especially *chaotic dynamics*, refer to Devaney [129]. Finally, a comprehensive treatment of probability theory can be found in Stark and Woods [530]. Nonlinear optimization techniques are detailed in Scales book on the subject [505].

More recently we are beginning to see the emergence of a class of neural network models that employ neurons that model closely the pulsed nature of action potentials and their integration through dendritic processing [275, 306, 363]. Such neurons employ *pulse-coded* signal functions [323] which assume that the information is contained in pulse frequencies that are sampled in a specific sampling interval. Networks of such neurons are called *pulse-coded neural networks*. Two fundamental properties of pulsed neurons are the use of pulses as signals, and the use of pulse products as a coupling mechanism. We defer any further discussion on pulsed neuron networks to Chapter 13 where we review this class of networks in detail.

Review Questions

- 3.1 Mention three important properties that neuronal signal functions should possess.
- 3.2 Show that if $S(x) = \frac{1}{1 + e^{-4x}}$ then $S^{-1}(\cdot)$ is monotonic increasing. Plot the inverse signal function.



- 3.3 Consider the neuronal signal function $\delta(x) = \tanh(cx)$. Show that $\delta(\cdot)$ is monotonic increasing. What can you say about the monotonicity of $\delta^{-1}(\cdot)$?
- 3.4 A sigmoidal neuron with $\lambda = 2$ has two input weights 0.5 and 0.75. For an input $X = (-1, 0.5)$ find the value of the bias weight such that the neuron output is 0.8.
- 3.5 Repeat Question 3.4 for
- Linear threshold signal function with $\alpha = 1$
 - Gaussian signal function with $\sigma = 2, c = 0$
- 3.6 For the matrix given below, find the eigenvalues and eigenvectors.

$$M = \begin{pmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{pmatrix}$$

What happens to the eigenvalues if the diagonal is zeroed out?

- 3.7 Convolve the functions $f(x) = u(x - 1)$ and $g(x) = \exp(-x^2)$. Plot the result. For your ready reference, the convolution integral is defined as

$$h(x) = \int_{-\infty}^{+\infty} g(\tau) f(x - \tau) d\tau$$

Provide a physical interpretation for convolution.

- 3.8 Consider the following probability density function:

$$p(x) = \begin{cases} 0 & x < 0 \\ \exp(x) & 0 \leq x \leq 2 \\ 0 & \text{otherwise} \end{cases}$$

Find:

- the mean or expectation of x : $\bar{x} = E[x]$
 - the variance of x : $\sigma^2 = E[(x - \bar{x})^2]$
- 3.9 What is the physical interpretation of mean and variance of a data scatter. Suppose we have two data scatters with the same means and different variances 0.1 and 5, what can we say about the nature of the data.
- 3.10 For a symmetric matrix $A \in \mathbb{R}^{n \times n}$, prove:
- The eigenvectors are orthogonal.
 - The eigenvectors for both A and A^{-1} are the same.
- 3.11 Multiplication of a matrix $A \in \mathbb{R}^{n \times n}$ into any vector $X \in \mathbb{R}^n$ rotates the point X in space. Check that with any 2×2 matrix of your choice. What happens to the rotation if the vector X is an eigenvector of A ?
- 3.12 In the context of neural networks, what do you understand by the following terms:
- graceful degradation
 - fault tolerance

- (c) cross-talk
 - (d) model-free estimation
- 3.13 Prove: A multilayer linear neuronal feedforward network is computationally equivalent to a single layer linear neural network. (By linear we mean that all neurons have purely linear signal functions.)
- 3.14 Most of the signal functions described above map activations into the interval $[0, 1]$: $S(x_j): x_j \rightarrow [0, 1]$. Design new versions of linear-threshold and logistic signal functions that map activations into the interval $[-1, 1]$: $S(x_j): x_j \rightarrow [-1, 1]$. Such neurons are said to be *bipolar* in nature since the signals can become both positive and negative.
- 3.15 Extend the one-dimensional Gaussian signal function (Eq. 3.14) to the n -dimensional case. Conjure up a physical interpretation for this.
- 3.16 Generate different versions of signal functions and play around with their parameters (λ , θ , and α) to get an idea of how these parameters relate to the shape and position of the signal functions. To get you started, here's the code segment for Fig. 3.2(a):

```
figure;
x = -10:0.1:10;
for i = 1:201
    if x(i) >= 0 out(i)=1;
    else out(i)=0;
    end
end
plot(x,out);
grid on;
axis([-10 10 -0.25 1.2]);
end
```

MATLAB

- 3.17 Assume a uniform distribution of points on the unit circle in two dimensions. You might choose points at an angular separation of 5° . Consider a neuron with a weight vector $(-0.25, 0.75)$, and no threshold.
- (a) Plot the inner product generated for each of the inputs against the angle the vector input makes with the x -axis, moving counter-clockwise from 0° through 360° .
 - (b) Plot neuron signal values against the angles 0° through 360° assuming that the signal function is binary threshold, linear threshold, and sigmoidal.
 - (c) Summarize your observations.